CHƯƠNG 5

Tích hợp AI tạo sinh vào vòng đời phát triển phần mềm

[Ứng dụng Trí tuệ nhân tạo]

[Nguyễn Tuấn Anh]
• [Thái Nguyên, 2026]



Giới thiệu nội dung

Mục tiêu chương

- Mô tả được cách AI tạo sinh tham gia vào từng nhóm hoạt động của SDLC: phân tích & thiết kế, triển khai, tích hợp & kiểm thử, vận hành & bảo trì.
- Phân tích được lợi ích, rủi ro và thách thức khi tích hợp AI vào quy trình phát triển phần mềm trên các khía cạnh năng suất, chất lượng, sáng tạo và hiệu quả.

Nội dung chính

1. Tích hợp AI trong các giai đoạn SDLC
2. Lợi ích, rủi ro và thách thức
3. Thiết kế workflow AI-augmented SDLC
4. AI Agent và Agentic AI
5. Thực hành

Khởi động: AI đã tham gia vào SDLC như thế nào?

- Năm 2026, một kỹ sư phần mềm mở IDE, nhập mô tả API endpoint – trong vài giây, AI sinh toàn bộ phần khung, kèm xử lý lỗi và comment. Khi CI/CD phát hiện lỗi bảo mật, AI phân tích log và đề xuất bản vá.
- **Khảo sát DORA 2024**: 75,9% lập trình viên đã sử dụng AI trong ít nhất một tác vụ công việc hàng ngày.
- **Khảo sát Accenture 2/2025**: 89% kỹ sư khẳng định kết quả công việc được cải thiện khi dùng GenAI.

Thực trạng

Phần lớn công cụ AI thương mại (Copilot, Cursor) mới chỉ tập trung vào **một lát cắt hẹp** – thường là viết mã hoặc gỡ lỗi. Tham gia AI **xuyên suốt toàn bộ vòng đời** mới mở khóa toàn bộ lợi ích về hiệu suất và chất lượng.

Bốn nhóm hoạt động chính trong SDLC dưới góc nhìn AI

1. **Phân tích và Thiết kế** (Analysis & Design): thu thập và phân tích yêu cầu, thiết kế kiến trúc hệ thống, thiết kế UI/UX.
2. **Triển khai** (Implementation): sinh mã nguồn, gợi ý mã, giải thích mã và các hoạt động lập trình.
3. **Tích hợp và Kiểm thử** (Integration & Testing): CI/CD, sinh ca kiểm thử, kiểm thử tự động, phân tích bảo mật.
4. **Vận hành và Bảo trì** (Operations & Maintenance): đánh giá mã, sửa lỗi, tối ưu hóa, giám sát và hỗ trợ vận hành.

Nguồn: Accenture & DFKI, 2025

Bốn nhóm này **không tách biệt hoàn toàn** – có sự giao thoa và phản hồi liên tục, đặc biệt trong môi trường agile và DevOps.

AI trong Phân tích và Thiết kế

- **Phân tích yêu cầu**: LLM đóng vai trò như *business analyst* hỗ trợ – phân tích ghi chú từ các bên liên quan, phát hiện yêu cầu **mâu thuẫn, thiếu sót hoặc mơ hồ**, đề xuất KPI đo lường được.
- **Thiết kế kiến trúc**: AI hỗ trợ đánh giá và so sánh phương án thiết kế – phân tích thư viện/framework dựa trên hiệu năng, khả năng mở rộng, mức độ bảo trì. Phát hiện **phản mẫu thiết kế** (design anti-patterns) từ sớm.
- **Thiết kế UI/UX**: AI tạo mẫu giao diện nhanh, đề xuất cải thiện trải nghiệm người dùng, sinh mã giao diện từ mô tả tự nhiên. Công cụ: Uizard, Figma AI.

Lưu ý

Đây là giai đoạn thường bị bỏ qua khi nói về AI trong SDLC, nhưng lại là nơi AI có thể tạo ra **tác động sâu sắc nhất** – quyết định ở đây ảnh hưởng đến toàn bộ phần còn lại của dự án.

AI trong Triển khai

- **Sinh mã nguồn** (Code Generation): GitHub Copilot giúp hoàn thành tác vụ nhanh hơn **55,8%**. Amazon Q Developer: nhanh hơn **57%**, khả năng thành công cao hơn **27%**. Đặc biệt hiệu quả với mã khung (boilerplate).
- **Gợi ý mã** (Code Suggestion): hoạt động như *pair programmer* luôn sẵn sàng – dự đoán và đề xuất phần tiếp theo dựa trên ngữ cảnh. Hữu ích khi làm việc với API không quen thuộc.
- **Giải thích mã** (Code Explanation): LLM đọc và giải thích mã nguồn phức tạp bằng ngôn ngữ tự nhiên, rút ngắn thời gian onboarding, giảm rào cản với legacy code.

Đây là nhóm hoạt động nhận được **nhiều sự chú ý nhất** và cũng là nơi các công cụ AI **trưởng thành nhất** hiện nay.

AI trong Tích hợp và Kiểm thử

- **Tích hợp hệ thống**: AI hỗ trợ sinh tệp cấu hình triển khai (Kubernetes manifests, Docker Compose), tự động hóa CI/CD.
  - Cảnh báo: khoảng 35,8% Kubernetes manifest do ChatGPT tạo chứa ít nhất một "mùi" (smell) chất lượng.
- **Sinh ca kiểm thử** (Test Generation): AI tự động sinh ca kiểm thử đơn vị, tích hợp và kiểm thử biên – bao gồm các trường hợp con người thường bỏ sót. Có thể sinh **dữ liệu kiểm thử tổng hợp** cho điều kiện biên.
- **Phân tích bảo mật**: AI quét mã nguồn tìm lỗ hổng, sinh báo cáo phân tích và đề xuất biện pháp khắc phục.

Lưu ý

Khoảng **40%** chương trình do Copilot tạo ra chứa lỗ hổng bảo mật (Pearce et al., 2025). Cần tích hợp công cụ quét bảo mật tự động và duy trì **đánh giá mã bởi con người**.

AI trong Vận hành và Bảo trì

- **Đánh giá mã nguồn** (Code Review): AI tự động phân tích pull request, gắn cờ vấn đề tiềm ẩn, đề xuất cải thiện. Giảm thời gian đánh giá, tăng tỉ lệ chấp nhận.
- **Sửa lỗi** (Program Repair): AI mô phỏng quy trình gỡ lỗi kiểu con người qua suy luận lặp. GitHub Copilot Chat giúp giảm thời gian sửa lỗi tới **37%** trong môi trường làm việc nhóm.
- **Tối ưu hóa và bảo trì**: AI đề xuất tái cấu trúc, phát hiện mã trùng lặp, tối ưu hiệu năng. Google: GenAI cho phép di chuyển mã hiệu quả hơn qua kết hợp AST + heuristic + LLM.
- **Giám sát**: AI tích hợp vào khung giám sát để phát hiện bất thường và đề xuất biện pháp khắc phục trước khi SLA bị vi phạm.

Bức tranh tổng thể: AI xuyên suốt SDLC

Nhóm hoạt động	Tác vụ chính	Vai trò của AI	Công cụ
Phân tích & Thiết kế	Phân tích yêu cầu, kiến trúc, UI/UX	Phát hiện mâu thuẫn, đề xuất kiến trúc, so sánh công nghệ	ChatGPT, Uizard
Triển khai	Sinh mã, gợi ý mã, giải thích mã	Sinh mã từ mô tả, tự động hoàn thiện, giải thích codebase	GitHub Claude C Q
Tích hợp & Kiểm thử	CI/CD, kiểm thử, phân tích bảo mật	Sinh ca kiểm thử, dữ liệu tổng hợp, quét lỗ hổng	Copilot, Tricentis
Vận hành & Bảo trì	Đánh giá mã, sửa lỗi, giám sát	Phân tích PR, đề xuất sửa lỗi, phát hiện bất thường	Atlassian GitLab Du

Điểm mấu chốt

Khi ngữ cảnh dự án được duy trì và chuyển tiếp qua từng pha, giá trị AI mang lại **không chỉ cộng dồn mà còn được nhân lên** – mỗi giai đoạn hưởng lợi từ hiểu biết đã được AI nắm bắt ở các giai đoạn trước.

Tạm dừng – Tự kiểm tra Section 1

Những điểm cần nhớ

1. AI tham gia vào **cả bốn nhóm hoạt động** của SDLC, không chỉ giai đoạn viết mã.
2. Mỗi nhóm có các tác vụ cụ thể được AI hỗ trợ – từ phân tích yêu cầu đến giám sát vận hành.
3. **Hiệu ứng tích lũy**: giá trị lớn nhất đến từ việc AI tham gia **liên tục** qua các pha, không phải từng pha riêng lẻ.

Tự kiểm tra nhanh:

1. Liệt kê bốn nhóm hoạt động SDLC. Với mỗi nhóm, nêu một tác vụ AI hỗ trợ.
2. Tại sao giai đoạn Phân tích & Thiết kế lại quan trọng dù thường bị bỏ qua?
3. Tỉ lệ mã do AI sinh chứa lỗ hổng bảo mật là bao nhiêu? Biện pháp

Khởi động: Con dao hai lưỡi

- **GitClear 2024**: dữ liệu từ hàng triệu thay đổi mã nguồn cho thấy trợ lý lập trình AI đang góp phần làm **suy giảm chất lượng mã** – tỉ lệ code churn gia tăng đáng kể.
- **METR 2025**: thử nghiệm có kiểm soát cho thấy lập trình viên mã nguồn mở có kinh nghiệm dùng AI mất **nhiều thời gian hơn 19%**.

Những phát hiện này dường như **mâu thuẫn** với các báo cáo trước đó về Copilot (nhanh hơn 55%) hay CodeWhisperer (nhanh hơn 57%).

Thực tế: tác động của AI **phụ thuộc** vào loại tác vụ, kinh nghiệm người dùng, chất lượng công cụ và **cách tổ chức tích hợp AI** vào quy trình.

Lợi ích (1): Năng suất và Hiệu quả

- **Năng suất** (Productivity) – làm được nhiều hơn với cùng nguồn lực:
  - Copilot: hoàn thành tác vụ nhanh hơn **55,8%** (2,7 giờ $\rightarrow$ 1,2 giờ).
  - Amazon Q Developer: nhanh hơn 57%, khả năng thành công cao hơn 27%.
  - 88% lập trình viên cảm thấy năng suất hơn; 87% tiết kiệm năng lượng tinh thần.
  - Giảm thời gian viết boilerplate tới **96%**.
- **Hiệu quả** (Efficiency) – đạt cùng kết quả với ít nguồn lực hơn:
  - Tự động hóa tác vụ lặp lại, giảm thời gian tìm kiếm thông tin.
  - Giảm chi phí chuyển đổi ngữ cảnh (context switching).

Phân biệt

Năng suất = **tăng đầu ra**. Hiệu quả = **giảm đầu vào**.

Lợi ích (2): Chất lượng

- **Cải thiện độ bao phủ kiểm thử**: AI đề xuất ca kiểm thử đơn vị và tích hợp, bao gồm các **trường hợp biên** con người thường bỏ qua.
- **Phát hiện lỗi sớm**: công cụ AI phát hiện lỗi tiềm ẩn ngay trong quá trình viết mã, trước khi lan sang các giai đoạn sau – nơi chi phí sửa lỗi **cao hơn nhiều**.
- **Đánh giá mã nhất quán**: AI không bị mệt mỏi hay thiên kiến như con người, duy trì chất lượng đánh giá đồng đều qua thời gian.
- **Cải thiện tài liệu**: AI tự động sinh và cập nhật tài liệu kỹ thuật từ mã nguồn.

Chiến lược mô hình phân tầng

Mô hình suy luận cao (o1/o3) cho tác vụ chính xác; mô hình đa năng (GPT-4o) cho sáng tạo và lặp nhanh. Phối hợp nhiều mô hình có thể **giảm tỉ lệ lỗi tới 23%** trong ứng dụng doanh nghiệp.

Lợi ích (3): Sáng tạo

- **Đề xuất phương án thay thế**: AI đề xuất nhiều cách tiếp cận khác nhau cho cùng bài toán, giúp lập trình viên mở rộng **không gian giải pháp**.
- **Khám phá nhanh**: khả năng sinh prototype trong vài giây cho phép thử nghiệm nhiều ý tưởng hơn trong cùng thời gian.
- **Kết hợp tri thức liên ngành**: AI mang kiến thức từ các lĩnh vực khác vào bài toán, tạo kết nối con người khó tự nhận ra.
- **Giải phóng thời gian cho tư duy chiến lược**: 27% lập trình viên tái đầu tư thời gian tiết kiệm được vào tư duy chiến lược hoặc tái cấu trúc (Stack Overflow, 2024).

Lưu ý

Tác động của AI đối với sáng tạo vẫn cần nghiên cứu thêm. AI có thể **vừa mở rộng vừa giới hạn** sáng tạo, tùy cách sử dụng.

Rủi ro (1): Hallucination và Bảo mật

- **Hallucination** (ảo giác): LLM tạo phản hồi có vẻ hợp lý nhưng phi thực tế.
  - Biểu hiện trong lập trình: gọi hàm API không tồn tại, phát minh tham số cấu hình không có thật, mô tả sai hành vi thư viện.
  - Đây là rủi ro nền tảng, chưa có giải pháp triệt để.
- **Lỗ hổng bảo mật trong mã AI sinh**:
  - Khoảng **40%** chương trình do Copilot tạo ra chứa lỗ hổng bảo mật.
  - Phổ biến: SQL injection, XSS, hàm mật mã không an toàn, xử lý đầu vào sai.
  - Mã do AI tạo **trông có vẻ đúng** – lập trình viên dễ bỏ qua bước kiểm tra.

Rủi ro (2): Suy giảm chất lượng và Phụ thuộc

- **Suy giảm chất lượng mã và nợ kỹ thuật**:
  - Tỉ lệ **code churn** (mã bị viết lại/xóa ngay sau khi tạo) gia tăng – mã AI sinh thường không bền vững.
  - Lập trình viên có xu hướng sao chép mã AI mà không tái cấu trúc.
  - Đánh giá quá cao khả năng AI, trì hoãn việc kiểm tra $\rightarrow$ tích lũy **nợ kỹ thuật**.
- **Phụ thuộc quá mức và suy giảm kỹ năng**:
  - Chấp nhận đề xuất AI mà không thực sự hiểu $\rightarrow$ suy giảm kỹ năng nền tảng.
  - "Lập trình viên quen dùng AI quên mất cách tự giải quyết vấn đề."
  - Đặc biệt nguy hiểm với lập trình viên mới vào nghề.

Rủi ro (3): Sở hữu trí tuệ và Quyền riêng tư

- **Bản quyền mã nguồn**: ai sở hữu mã do AI tạo ra? Mã có vi phạm bản quyền của mã trong tập huấn luyện không?
- **Lộ dữ liệu nhạy cảm**: khi gửi mã nguồn nội bộ lên dịch vụ AI đám mây, dữ liệu độc quyền có thể bị lộ.
- **Tuân thủ quy định**: GDPR, SOC 2 và các quy định khác cần được xem xét khi tích hợp AI.

Khuyến nghị

- Sử dụng **enterprise plan** hoặc **self-hosted instance** cho dự án nhạy cảm.
- Thiết lập quy trình kiểm tra bản quyền trước khi đưa mã AI sinh vào sản phẩm thương mại.
- **Không gửi** dữ liệu khách hàng, khóa API, hoặc thông tin cá nhân lên dịch vụ AI công cộng.

Thách thức ở cấp độ tổ chức

- **Thích nghi của nhóm**: cần hướng dẫn rõ ràng về thời điểm và cách dùng AI; điều chỉnh tiêu chuẩn lập trình cho mã AI tạo; cân bằng giữa hỗ trợ AI và giám sát con người.
- **Niềm tin và chấp nhận công cụ**: nhiều lập trình viên mất niềm tin sau sai lầm ban đầu của AI. Chất lượng, giá trị chức năng và khả năng duy trì mục tiêu là yếu tố quyết định.
- **Phân tích chi phí – lợi ích**:
  - Chi phí thiết lập và đào tạo ban đầu có thể đáng kể.
  - Lợi ích năng suất theo thời gian bù đắp chi phí.
  - Cần đánh giá **ROI** cho từng điểm tích hợp AI.

McKinsey: 18% tổ chức dùng GenAI cho kỹ nghệ phần mềm; riêng lĩnh vực công nghệ là **36%**. Hiệu quả AI thay đổi theo độ phức tạp tác vụ.

Khởi động: Không chỉ là “thêm AI vào”

- Nhiều tổ chức tiếp cận AI đơn giản: mua license Copilot cho toàn đội ngũ và kỳ vọng năng suất tăng vọt.
- Cách tiếp cận **“thêm AI vào”** (add AI on top) thường thất bại trong việc tạo giá trị bền vững.

Sự thật quan trọng

AI không phải là một **plugin** cắm vào quy trình cũ – nó là một thành phần làm **thay đổi chính quy trình đó**. Thiết kế AI-augmented SDLC đòi hỏi suy nghĩ lại về vai trò của con người và máy móc trong từng giai đoạn.

Nguyên tắc 1–2: AI xuyên suốt + Con người giám sát

1. **AI tham gia xuyên suốt, không chỉ ở giai đoạn viết mã**
   - Hầu hết công cụ thương mại mới tập trung vào lập trình/gỡ lỗi – cách tiếp cận "đảo biệt lập" (siloed).
   - AI-augmented SDLC đích thực: AI tham gia từ **ý tưởng đến ứng phó sự cố**, với ngữ cảnh dự án được duy trì liên tục.
   - Hymel (2024): lập trình viên chuyển từ **người sáng tạo chủ động** sang **người xác nhận và điều phối**.
2. **Con người giám sát, AI thực thi** (Human-in-the-Loop)
   - AI đảm nhận tác vụ kỹ thuật, lặp lại, dễ mắc lỗi; con người tập trung vào xác định bài toán, phác thảo giải pháp, điều phối và kiểm tra phản biện.
   - Treude & Gerosa (2025): 11 chế độ tương tác lập trình viên–AI, chuyển đổi linh hoạt giữa chấp nhận, chỉnh sửa hoặc từ chối đề xuất.

Nguyên tắc 3–4: Mô hình phân tầng + Duy trì ngữ cảnh

3. **Sử dụng mô hình phân tầng** (Tiered Model Utilization)
   - **Mô hình đa năng** (GPT-4o, Claude Sonnet): sáng tạo, brainstorming, lặp nhanh, tương tác tự nhiên.
   - **Mô hình suy luận cao** (o1/o3, Claude Opus): logic phức tạp, xử lý trường hợp biên, tác vụ đòi hỏi độ chính xác cao.
   - **Mô hình chuyên biệt**: tinh chỉnh cho tác vụ cụ thể (Codex, DeepSeek Coder cho sinh mã).
4. **Duy trì và chuyển tiếp ngữ cảnh**
   - **Tổng kết định kỳ**: tóm tắt ngữ cảnh thành bản bàn giao, bắt đầu phiên mới.
   - **Phân mảnh luồng**: duy trì nhiều phiên song song, mỗi phiên một chủ đề con.
   - **Kích hoạt tự phản ánh**: định kỳ hỏi mô hình về mức độ bão hòa cửa sổ chú ý.

Case study: Tối ưu hóa định tuyến kho hàng

- **Bài toán**: "Là nhà phân tích logistics, tôi cần thuật toán tối ưu hóa định tuyến giảm thiểu thời gian lấy hàng trong kho."
- **Cách truyền thống**: nhóm dành hàng giờ so sánh thư viện, lần lượt tạo mẫu với từng lựa chọn.
- **Với LLM**: thời gian nghiên cứu rút xuống còn **vài phút**, thu được kiến thức mới qua đối thoại trực tiếp.

Môi trường thí nghiệm – chiến lược phối hợp 3 mô hình:

- **GPT-4o**: đối tác chính cho brainstorming, tạo ý tưởng nhanh, đối thoại giọng nói và đầu vào hình ảnh.
- **GPT-4.1** (1M token context): cho phiên lập trình dày đặc hướng dẫn, mã nguồn lớn.
- **o3/o4-mini**: suy luận chính xác, tinh chỉnh thuật toán, thiết kế hàm chi phí.

Case study: Các pha thực hiện

1. **Pha 1 – Phân tích yêu cầu** (GPT-4o): AI xác định KPI, liệt kê ràng buộc (lối đi một chiều, chướng ngại vật động), đề xuất chỉ số thành công.
2. **Pha 2 – Đánh giá công nghệ** (GPT-4o): quét và so sánh NetworkX, SciPy, OR-Tools trong vài phút – thay vì hàng giờ nghiên cứu thủ công.
3. **Pha 3 – Tạo mẫu nhanh** (phối hợp 3 mô hình): AI sinh mã khung, khung kiểm thử và tài liệu nội tuyến trong **chưa đầy 30 giây** từ một prompt duy nhất.
4. **Pha 4 – Kiểm thử và tối ưu** (GPT-4o): tự động tạo bộ kiểm thử toàn diện, dữ liệu tổng hợp, property-based testing.
5. **Pha 5 – Tài liệu** (GPT-4.1): tạo tài liệu API, hướng dẫn, sơ đồ kiến trúc, đồ thị tri thức (knowledge graph).
6. **Pha 6 – Triển khai và cải tiến** (GPT-4.1 + GPT-4o): giám sát, phát hiện bất thường, phân tích liên tục để tinh chỉnh thuật toán.

Hiệu ứng ngữ cảnh tích lũy & Từ lý thuyết đến thực hành

- **Compounding Context Effect**: khi AI tham gia liên tục qua tất cả các pha, ngữ cảnh từ pha trước được kết hợp và làm giàu ở pha sau $\rightarrow$ lợi ích **vượt xa tổng đơn giản** của từng phần.
- Lập trình viên báo cáo: rõ ràng cao hơn, onboarding nhanh hơn với thư viện lạ, giảm chi phí chuyển đổi ngữ cảnh.

Bài học thực tiễn

Case study được thực hiện **hoàn toàn trong giao diện web ChatGPT** – không cần nền tảng tích hợp toàn diện. Các nguyên tắc thiết kế có thể được áp dụng **ngay hôm nay** với công cụ sẵn có, miễn là có chiến lược rõ ràng về cách AI tham gia và cách duy trì ngữ cảnh.

Từ LLM đến AI Agent

- Cho đến nay, ta chủ yếu nói về AI như **công cụ phản ứng**: đưa prompt $\rightarrow$ nhận phản hồi.
- Nhưng hãy tưởng tượng: giao cho AI một mục tiêu cấp cao – "phân tích yêu cầu, thiết kế CSDL, viết API, tạo frontend, viết kiểm thử, triển khai" – và AI **tự động lập kế hoạch, thực thi, kiểm tra và sửa lỗi**.
- Đây là thế giới của **AI Agent**: AI không chỉ sinh nội dung theo yêu cầu, mà còn có khả năng **tự chủ** trong lập kế hoạch, ra quyết định, sử dụng công cụ và phối hợp đa tác tử.

Sự tiến hóa:

- **LLM** = "bộ não"
- **AI Agent** = "bộ não + khả năng hành động"
- **Agentic AI** = "bộ não + khả năng hành động + khả năng tự tổ chức và thích ứng"

Non-agentic vs Agentic workflow

Non-agentic workflow

- AI hoàn thành tác vụ trong **một lần duy nhất**, không lặp lại.
- Ví dụ: yêu cầu viết hàm Python $\rightarrow$ AI trả về code và kết thúc.
- Đầu ra được tạo **one-shot**, không xem lại hay chỉnh sửa.

Agentic workflow

- AI hoạt động theo **vòng lặp**: tạo – tự phê bình – chỉnh sửa – đánh giá lại – tiếp tục.
- Mô phỏng cách con người giải quyết vấn đề: viết nháp, đọc lại, sửa, kiểm tra.

Kết quả benchmark

HumanEval (OpenAI) liên tục cho thấy **agentic workflow vượt trội**

Bốn mẫu nền tảng (1): Phản tư và Sử dụng công cụ

1. **Phản tư** (Reflection)
   - LLM tự phê bình và tinh chỉnh đầu ra của chính mình qua nhiều vòng lặp.
   - Trong lập trình: tự phát hiện và sửa lỗi biên dịch/logic; cải thiện chất lượng mã; đảm bảo tuân thủ tiêu chuẩn.
   - Ví dụ: sau khi ChatGPT tạo code, yêu cầu mô hình xác định lỗi hoặc đề xuất cải tiến – thường phát hiện lỗi lần đầu bỏ sót.
2. **Sử dụng công cụ** (Tool Use / API Calls)
   - LLM tương tác với hệ thống bên ngoài: gọi API, truy vấn CSDL, chạy lệnh build/test.
   - **MCP** (Model Context Protocol – Anthropic): kết nối AI với nguồn dữ liệu qua giao thức duy nhất.
   - **A2A** (Agent-to-Agent – Google): cho phép agent từ các nền tảng khác nhau giao tiếp.
   - Trong SDLC: tương tác git/GitHub, chạy CI/CD, đọc/ghi tệp, thực thi lệnh hệ thống.

Bốn mẫu nền tảng (2): Lập kế hoạch và Cộng tác đa agent

3. **Lập kế hoạch** (Planning)
   - LLM phân rã chỉ thị cấp cao thành các bước nhỏ, thực thi theo trình tự logic.
   - Ví dụ xây dựng ứng dụng web: (1) phân tích yêu cầu, (2) thiết kế CSDL, (3) tạo cấu trúc backend, (4) triển khai logic, (5) tạo frontend, (6) viết kiểm thử, (7) cấu hình triển khai.
4. **Cộng tác đa agent** (Multi-agent Collaboration)
   - Nhiều LLM chuyên biệt làm việc song song hoặc theo pipeline.
   - Ví dụ hệ thống đa agent cho SDLC: Agent Phân tích, Agent Kiến trúc, Agent Lập trình, Agent Kiểm thử, Agent Bảo mật, Agent Đánh giá, Agent Điều phối (Orchestrator).
   - Mẫu mạnh mẽ nhất nhưng cũng phức tạp nhất.

Các nền tảng và công cụ Agentic AI

Framework	Đặc điểm chính	Thế mạnh
LangChain / Lang-Graph	Workflow dạng đồ thị, đa agent	Tích hợp vendor rộng, hệ phong phú
AutoGen (Microsoft)	Đa agent, trừu tượng hóa hội thoại cấp cao	Tối ưu chi phí, đa agent linh
CrewAI	Framework đa agent dựa trên workflow	Dễ sử dụng, nhiều vendor LL
Semantic Kernel (MS)	SDK tích hợp LLM + kho dữ liệu; Pro-cess Framework	Tích hợp sâu hệ sinh thái Mi
Google ADK	Module, linh hoạt; tích hợp MCP và A2A	Model-agnostic, deployment-
PydanticAI	Type-safety, quan sát thời gian thực, dependency injection	Đơn giản, an toàn kiểu

Công cụ phát triển agentic

Claude Code (Anthropic): công cụ dòng lệnh agentic. **Devin** (Cognition AI): coding agent tự động. **Cursor, Windsurf**: IDE agentic tích hợp AI đa bước.

AI Agent trong thực hành & Chiến lược áp dụng

Các kịch bản ứng dụng điển hình:

- Tự động hóa kiểm thử | Đánh giá mã tự động | Sửa lỗi tự động
- Di chuyển mã và hiện đại hóa | Sinh tài liệu | Hỗ trợ DevOps

Chiến lược áp dụng theo giai đoạn:

1. **Agent đơn**: một agent cho tác vụ phạm vi rõ ràng, rủi ro thấp – xây dựng niềm tin.
2. **Agent chuyên biệt**: nhiều agent, mỗi agent một vai trò (planner, executor, critic).
3. **Cộng tác đa agent**: pipeline đầy đủ từ phân tích yêu cầu đến triển khai.
4. **Agentic AI toàn diện**: hệ thống tự tổ chức, tự thích ứng, liên tục cải tiến.

Lưu ý

Ở mọi giai đoạn: thiết lập chỉ số thành công rõ ràng, giám sát tương tác và chi phí điều phối, liên tục đánh giá và điều chỉnh.

Rủi ro và cân nhắc khi sử dụng AI Agent

- **Hành vi không mong muốn**: hệ thống đa agent có thể biểu hiện **hành vi nảy sinh** (emergent behaviors) không được dự đoán trước.
- **Khó khăn trong gỡ lỗi**: truy vết nguyên nhân qua nhiều agent và tương tác phức tạp hơn nhiều so với hệ thống đơn agent.
- **Chi phí**: mỗi agent tiêu thụ token và tài nguyên – chi phí tăng nhanh theo số lượng agent và vòng lặp.
- **Bảo mật**: agent có quyền truy cập công cụ và hệ thống bên ngoài $\rightarrow$ vector tấn công mới.
- **Trách nhiệm giải trình**: khi hệ thống đa agent quyết định sai – **ai hoặc cái gì chịu trách nhiệm**?

Nguyên tắc vàng

Con người phải duy trì vai trò giám sát cuối cùng, đặc biệt với các quyết định ảnh hưởng đến bảo mật, an toàn hoặc tuân thủ. Tích hợp từng bước và thiết lập cơ chế giám sát sớm.

Thực hành: Thiết kế AI-augmented workflow

Mục tiêu

Xây dựng quy trình phát triển phần mềm có AI hỗ trợ cho hệ thống Quản lý bán hàng (tiếp tục từ Bài 4). AI được dùng để hỗ trợ sinh mã, kiểm tra chất lượng, phát hiện lỗi, sinh tài liệu và đề xuất giải pháp kỹ thuật.

Sáu bước thực hiện:

1. **Mô tả quy trình hiện tại**: vẽ quy trình từ tiếp nhận yêu cầu đến phát hành; xác định điểm tốn thời gian hoặc thường phát sinh lỗi.
2. **Chọn điểm tích hợp AI**: làm rõ yêu cầu, thiết kế, lập trình, rà soát, kiểm thử, tài liệu. Không dùng AI cho bước không thể kiểm chứng kết quả.
3. **Thiết kế cơ chế kiểm soát**: thay đổi do AI tạo phải được đọc, kiểm thử và rà soát trước khi hợp nhất.

Thực hành: Các bước thực hiện (tiếp theo)

4. **Thử nghiệm trên một chức năng mới**: thực hiện theo quy trình có AI; ghi thời gian, số lỗi, số lần hỏi lại, tỉ lệ nội dung phải sửa.
5. **So sánh và đánh giá**: so sánh với nhiệm vụ tương tự trước đây; đánh giá lợi ích, chi phí, rủi ro, độ tin cậy. Phân biệt **tiết kiệm thời gian tạo bản nháp** với **tổng thời gian để có sản phẩm đạt chuẩn**.
6. **Chuẩn hóa quy trình**: cập nhật sơ đồ, checklist và trách nhiệm; xác định trường hợp được dùng/không được dùng AI; đề xuất chỉ số theo dõi.

Lưu ý quan trọng

- Không dùng số liệu do AI tự ước lượng làm kết quả đo thực nghiệm.
- Mọi thay đổi phải **truy vết được** về nhiệm vụ và người phê duyệt.
- Đánh giá cả lỗi AI không phát hiện và lỗi do AI tạo thêm.

Thực hành: Prompt mẫu & Sản phẩm cần nộp

Prompt mẫu 1 – Phân tích điểm tích hợp:

"Dựa trên quy trình phát triển sau: [mô tả], hãy đề xuất các điểm có thể dùng AI. Với mỗi điểm, trình bày mục tiêu, đầu vào, đầu ra, lợi ích dự kiến, rủi ro, cách kiểm chứng và vai trò chịu trách nhiệm."

Prompt mẫu 2 – Rà soát thay đổi:

"Hãy rà soát thay đổi sau cho nhiệm vụ [mô tả] theo yêu cầu [tiêu chí]. Kiểm tra tính đúng, phạm vi, bảo mật, xử lý lỗi, khả năng bảo trì, kiểm thử và tài liệu."

Sản phẩm cần nộp:

- Sơ đồ quy trình trước và sau tích hợp AI + Ma trận điểm tích hợp
- Một chức năng mới thực hiện theo quy trình + Báo cáo số liệu thử nghiệm
- Checklist chuẩn và Nhật ký sử dụng AI của Bài 5

Kết nối với dự án môn học

Dự án: Trợ lý phát triển phần mềm thông minh

Sinh viên sẽ áp dụng trực tiếp các khái niệm về AI Agent trong chương này:

- **Thiết kế và triển khai** ít nhất một agent chuyên biệt cho một tác vụ SDLC (agent sinh kiểm thử, agent đánh giá mã, agent sinh tài liệu).
- **Tích hợp agent** vào quy trình phát triển có AI hỗ trợ, tuân thủ các nguyên tắc thiết kế AI-augmented SDLC.
- **Đánh giá hiệu quả** dựa trên chỉ số cụ thể: thời gian tiết kiệm, tỉ lệ lỗi phát hiện, chất lượng đầu ra.
- **Ghi nhận thách thức** trong quá trình triển khai và đề xuất hướng cải tiến.

Dự án là cơ hội để trải nghiệm thực tế việc xây dựng và tích hợp AI

Tổng kết chương

Những điểm cần ghi nhớ

- **AI tham gia xuyên suốt SDLC** qua bốn nhóm hoạt động: Phân tích & Thiết kế, Triển khai, Tích hợp & Kiểm thử, Vận hành & Bảo trì. Giá trị lớn nhất đến từ **hiệu ứng ngữ cảnh tích lũy** khi AI tham gia liên tục qua các pha.
- **Lợi ích và rủi ro song hành:** AI tăng năng suất 50–100%, cải thiện chất lượng kiểm thử và mở rộng không gian giải pháp. Nhưng cũng mang đến hallucination, lỗ hổng bảo mật (khoảng 40% mã AI sinh), suy giảm chất lượng và rủi ro sở hữu trí tuệ – đòi hỏi **giám sát con người** chặt chẽ.
- **Thiết kế AI-augmented SDLC** dựa trên bốn nguyên tắc: AI xuyên suốt, con người giám sát (HITL), mô hình phân tầng và duy trì ngữ cảnh. Có thể áp dụng ngay với công cụ sẵn có nếu có chiến lược rõ ràng.
- **Agentic AI** với bốn mẫu nền tảng – Phản tư, Sử dụng công cụ